

座・Navi プログラム解説書（初学者向け）

このドキュメントは「座・Navi (Za-Navi)」のプログラムが、それぞれ何をしているのか・どこをどう直すと何が変わるのかをまとめたものです。チームメンバーが「自分はどのファイルを見ればいいか」「ここを直したら何が起きるか」を理解する手助けになることを目指しています。

1. 全体の流れ（MVCモデル2）

このアプリは「**MVCモデル2**」という設計で作られています。難しく聞こえますが、やっていることはシンプルです。

```
① ブラウザ
  ↓ 「予約確認のページが見たい」 (URL: /ControlServlet?action=reserveConfirm)
② ControlServlet (フロントコントローラ)
  ↓ 「予約確認の処理は ReserveConfirmLogic に任せよう」
③ ooLogic (ビジネスロジック = 処理担当)
  ↓ 「データが必要だから DAO に取りに行ってもらおう」
④ ooDAO (データアクセス = データ取得担当)
  ↓ 「はい、このデータどうぞ」 (今はモックデータ、将来はMySQL)
⑤ ooLogic
  ↓ 「このデータを画面に渡して、この画面を表示してね」
⑥ ControlServlet
  ↓ 「JSPに表示をお願いします」
⑦ oo.jsp (画面表示担当)
  ↓
⑧ ブラウザに画面が表示される
```

ポイント

- **画面（JSP）に複雑な処理を書かない** → 処理は全部 Logic に書く
 - **Logicにデータベースのことを書かない** → データの取得は全部 DAO に任せる
 - それぞれの役割が分かれているので、「画面のデザインを直したい人」「処理のルールを直したい人」「データの中身を直したい人」が、お互いのコードを壊さずに作業できる
-

2. フォルダ構成と役割

```
src/main/java/
├─ entity/    ... データの「入れ物」（クラス）。テーブルの1行に対応するイメージ
├─ dao/       ... データを取得・登録・更新・削除する係（今はモックデータ）
├─ model/     ... 画面ごとの処理担当（ビジネスロジック）
└─ control/   ... 全リクエストの窓口（フロントコントローラ）

src/main/webapp/
├─ WEB-INF/jsp/ ... 画面表示担当（JSP）
├─ css/         ... 見た目（スタイルシート）
└─ images/      ... アイコン画像
```

「直したいもの」から逆引き

やりたいこと	触るべきフォルダ・ファイル
ボタンの色や文字サイズなど見た目を変えたい	<code>webapp/css/style.css</code>
画面に表示する項目・文言を変えたい	<code>webapp/WEB-INF/jsp/oo.jsp</code>
アイコン画像を差し替えたい	<code>webapp/images/ico_oo.png</code> （同じファイル名で上書きすればJSP側の修正は不要）
ヘッダーのメニュー項目を増やしたい・減らしたい	<code>webapp/WEB-INF/jsp/common/header.jspf</code> （一般）/ <code>headerAdmin.jspf</code> （管理者）
「予約できる条件」など処理のルールを変えたい	<code>model/ooLogic.java</code>
表示するデータの中身（座席名・人数など）を変えたい	<code>dao/ooDAO.java</code>
新しい画面・新しい機能を追加したい	<code>entity</code> → <code>dao</code> → <code>model</code> → <code>control</code> → <code>jsp</code> の順に作る

3. 各レイヤーの詳しい説明

3-1. entity（エンティティ）＝データの入れ物

`entity/Account.java`、`entity/Reservation.java`、`entity/Seat.java`、`entity/Location.java` の4つがあります。

```
public class Account {
    private String userId;    // ユーザーID
    private String password; // パスワード
    private String name;     // 氏名
    private int bId;         // 部門ID
    private int admin;       // 管理者フラグ (1=管理者, 0=一般)

    // ... getter/setter ...
}
```

これは「データベースの1行を表すクラス」です。例えば `accounts` テーブルの1行（1人のユーザー情報）を、Javaのプログラムの中で扱いやすくするための「箱」だと考えてください。

ここを直すとき: 新しい項目（例: 電話番号）を増やしたいときは、entityにフィールドを追加 → DAOのモックデータにも値を追加 → 使う場所（Logic・JSP）でも追加、という流れになります。

3-2. dao（データアクセスオブジェクト）＝データの取得係

`dao/AccountsDAO.java` を例に見てみましょう。

```
public class AccountsDAO {
    private static final List<Account> ACCOUNTS = Arrays.asList(
        new Account("user001", "pass001", "田中太郎", 1, 0),
        new Account("admin001", "admin001", "管理者山田", 3, 1)
        // ...
    );

    // TODO: 後でMySQL接続に差し替え: SELECT * FROM accounts WHERE user_id = ?
    public Account findById(String userId) {
        return ACCOUNTS.stream()
            .filter(a -> a.getUserId().equals(userId))
            .findFirst().orElse(null);
    }
}
```

- 上の `Arrays.asList( ... )` の中身が「今はデータベースの代わりに使っている仮データ（モックデータ）」です
- `findById()` のようなメソッドが「データを取り出す処理」です。今はリストの中から探していますが、本来はここでSQLを実行してデータベースから取得します
- `// TODO: 後でMySQL接続に差し替え` というコメントがある場所が、**将来データベースに繋ぐときに書き換える場所**です（詳しくは [02 DB接続ガイド.md](#) を参照）

ここを直すとき: 「テスト用のユーザーを増やしたい」「座席の数を増やしたい」など、表示されるデータ自体を変えたいときは、この `Arrays.asList( ... )` の引数（カンマ区切りで並んでいる `new Account(...)` や `new Seat(...)` など）を編集します。

⚠ 注意: `Arrays.asList(...)` で作ったリストは「**サイズ変更不可 (fixed-size)**」です。あとから `.add(...)` や `.remove(...)` を呼ぶと `UnsupportedOperationException`（実行時エラー）になります。データを増やしたいときは、`.add()` するのではなく、`Arrays.asList(...)` の引数の中に直接 `new Account(...)` を1行追加するのが正しいやり方です（詳しくは次の「6. よくある『ここを直したい』Q&A」も参照）。

3-3. model（ロジック） = 画面ごとの処理担当

`model/Logic.java` というインターフェース（約束事）を、各画面の処理クラスが実装しています。

```
public interface Logic {  
    String execute(HttpServletRequest req, HttpServletResponse res);  
}
```

「`execute` という名前のメソッドを必ず作ってね、戻り値は次に表示するJSPのパス（文字列）にしてね」という約束です。

例えば `LoginLogic.java` :

```

public class LoginLogic implements Logic {
    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) {
        String userId = req.getParameter("userId");
        String password = req.getParameter("password");

        Account account = new AccountsDAO().findById(userId);

        if (account != null && account.getPassword().equals(password)) {
            // ログイン成功 → セッションに情報を保存して、メニュー画面へ
            HttpSession session = req.getSession();
            session.setAttribute("userId", account.getUserId());
            session.setAttribute("userName", account.getName());
            return account.getAdmin() == 1
                ? "/WEB-INF/jsp/adminMenu.jsp"
                : "/WEB-INF/jsp/menu.jsp";
        } else {
            // ログイン失敗 → エラーメッセージを画面にセットして、ログイン画面へ
            req.setAttribute("errorMsg", "IDまたはパスワードが正しくありません。");
            return "/WEB-INF/jsp/loginNG.jsp";
        }
    }
}

```

やっていることは3ステップです。

1. 画面から送られてきた値を受け取る（`req.getParameter(...)`）
2. DAOからデータを取得して、判定・加工する（業務のルール）
3. 画面に渡すデータをセットして（`req.setAttribute(...)`）、次に表示するJSPのパスを返す

ここを直すとき：「パスワードが3回間違ったらロックする」「土日は予約できないようにする」など、**業務のルール・条件**を変えたいときはここを編集します。

3-4. control（コントローラ）＝全リクエストの窓口

`control/ControlServlet.java` がアプリの「玄関」です。すべてのリクエストは `/ControlServlet?action=oo` という形でここに届きます。

```

switch (action) {
    case "login":
    case "doLogin":
        return new LoginLogic().execute(req, res);

    case "reserve":
    case "doReserve":
        return new ReserveLogic().execute(req, res);

    // ...
}

```

「`action=login` というリクエストが来たら `LoginLogic` に処理を任せる」という振り分け表になっています。

ここを直すとき: 新しい画面・機能を追加したときは、ここに `case` を1行追加して、対応する Logic を呼び出すようにします。

3-5. jsp（画面） = 表示担当

`webapp/WEB-INF/jsp/menu.jsp` などが画面の見た目を作っています。

```

<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>

...

<a href="<%= request.getContextPath() %>/ControlServlet?action=reserve" class="menu-btn">
    座席予約
</a>

```

- `<%@ page %>` は「このページの設定」を書く場所（文字コードなど）
- `<a href="...">` のリンクが押されると、また `ControlServlet` にリクエストが飛ぶ
- `<%= ... %>` の部分には、Logicが用意したデータ（`req.setAttribute(...)` したもの）を表示できます

ここを直すとき: 文言・レイアウト・表示する項目を変えたいときはここを編集します。ロジック（計算や判定）をJSPに書かないのがルールです。

4. 「画面が動くまで」の具体例（ログイン画面で確認）

1. ブラウザで `http://localhost:8080/za-navi/ControlServlet?action=login` を開く
2. `ControlServlet` が `action=login` を受け取り、`LoginLogic` を呼ぶ
3. `LoginLogic` は何もせず（GETなので）`/WEB-INF/jsp/login.jsp` を返す
4. `login.jsp` が表示される
5. ユーザーがID・パスワードを入力して送信 → 今度は `action=doLogin` というリクエストが飛ぶ
6. `ControlServlet` が `LoginLogic` を呼ぶ（同じLogicがログイン処理も担当）
7. `LoginLogic` が `AccountsDAO` からアカウント情報を取得し、ID・パスワードを照合
8. 成功なら `menu.jsp`（または `adminMenu.jsp`）、失敗なら `loginNG.jsp` を返す
9. `ControlServlet` がそのJSPに画面表示を依頼する

この「URLのaction → Logic → DAO → 結果に応じてJSPを選ぶ」という流れは、**すべての画面で共通**です。1つの画面の流れが理解できれば、他の画面も同じ読み方で理解できます。

5. 機能解説：実際のコードを読んでもみる

ここでは、実際にこのアプリに入っている「ちょっと工夫した処理」を3つ取り上げて、**コードがどう書いてあるか・どこを直せば動きが変わるか**を具体的に見ていきます。

5-1. ヘッダーのプルダウンメニュー（JavaScriptを使わない実装）

`webapp/WEB-INF/jsp/common/header.jspf` を開くと、メニューが次のように書かれています。

```
<details class="menu-dropdown">
  <summary>
     <%= session.getAttribute("userName") %> さん <span class="caret">▼</span>
  </summary>
  <div class="dropdown-panel">
    <a href="...?action=menu">メニュー</a>
    <a href="...?action=logout">ログアウト</a>
  </div>
</details>
```

`<details>` と `<summary>` はHTML標準のタグで、**JavaScriptを書かなくてもクリックで開閉するメニュー**が作れます。`<summary>` がクリックする部分（見出し）、その中の `<div class="dropdown-panel">` が開いたときに表示される中身です。見た目は `webapp/css/style.css` の `.menu-dropdown` `.dropdown-panel` で調整しています。

ここを直すとき: メニューの項目を増やしたい・並び替えたいときは、`<div class="dropdown-panel">` の中の `<a>` タグを編集します（一般ユーザー用は `header.jspf`、管理者用は `headerAdmin.jspf`）。

5-2. 行先登録で「出社」を選んだら座席予約画面へ進む

model/LocationRegistLogic.java の doRegist メソッドを見ると、行き先によって処理を分けています。

```
private String doRegist(HttpServletRequest req, HttpServletResponse res) throws Exception {
    String gyosaki = req.getParameter("gyosaki"); // 出社 / 在宅 / 出張
    if ("出張".equals(gyosaki)) {
        return "/WEB-INF/jsp/businessTrip.jsp";
    }
    if ("在宅".equals(gyosaki)) {
        return "/WEB-INF/jsp/remoteWork.jsp";
    }
    // 出社：ここでは登録せず座席予約画面（ReserveLogic）へリダイレクトし、
    // 座席が決まった時点で「出社+座席」をまとめて1件登録する
    // （そうしないと「出社（座席なし）」と「出社（座席あり）」の2件ができてしまうため）
    res.sendRedirect(req.getContextPath() + "/ControlServlet?action=reserve");
    return null;
}
```

ポイントは `res.sendRedirect(...)` と `return null` の組み合わせです。Logic インターフェース（model/Logic.java）の説明にある通り、`execute`（や内部メソッド）が `null` を返すのは「もう自分でリダイレクトしたので、ControlServlet は何もしなくてよい」という合図です。`return "/WEB-INF/jsp/xxx.jsp"` で**画面の切り替え（forward）をするのか、`res.sendRedirect(...)` + `return null` で別のactionへの飛び直し（redirect）**をするのか、の使い分けがここで学べます。

つまりポイント: 以前の版では「出社」を選んだ時点でこの `doRegist` 内でも1件登録していたため、座席予約画面で座席を選んだときの登録と合わせて「出社（座席なし）」「出社（座席あり）」の**2件が重複登録**されてしまうバグがありました。「出社」のときは登録せずリダイレクトだけする＝**登録は必ず1箇所（ReserveLogic）に集約する**、という設計が直した後の形です。

「在宅」「出張」も専用画面でメモを書いてから登録: どちらも一旦 `remoteWork.jsp` / `businessTrip.jsp` に forward し、利用者が任意でメモを入力してから `doRemoteWork` / `doBusinessTrip` で登録する流れになっています。登録後は `saveAndGoMenu` が `ControlServlet?action=menu`（または管理者なら `adminMenu`）へ `redirect` し、セッションの `isAdmin` を見てメニューを振り分けます（完了メッセージはリダイレクトをまたいで表示する必要があるため、リクエスト属性ではなくセッションの一時的な「フラッシュメッセージ」として持たせています）。

つまりポイント: LocationRegistLogic 側に新しい action (remoteWork / doRemoteWork) の処理を追加しても、入口の ControlServlet の switch に対応する case を足し忘れると、default 分岐でログイン画面に飛ばされてしまいます。**新しい action を追加するときは、必ず ControlServlet と各 Logic クラスの両方に手を入れる、**という基本を再確認できる典型例です。

ここを直すとき: 行き先ごとの遷移ルールを変えたいときはこの if 分岐を、登録後の戻り先や完了メッセージを変えたいときは saveAndGoMenu を編集します。

5-3. 検索結果に座席情報を追加する（DAOを組み合わせる例）

model/NameSearchLogic.java ・ DepartmentSearchLogic.java では、検索結果の各ユーザーについて「本日の予約」から「予約している座席」まで芋づる式に取得しています。

```
List<Reservation> todayRes = resDAO.findTodayByUserId(a.getUserId());
String todayStatus = "未登録";
Seat seat = null;
if (!todayRes.isEmpty()) {
    Reservation r = todayRes.get(0);
    todayStatus = r.getGyosaki();
    if (r.getSeatId() > 0) {
        seat = seatDAO.findById(r.getSeatId()); // 予約 → 座席IDから座席情報を取得
    }
}
row.put("todayStatus", todayStatus);
row.put("seat", seat); // JSPへ渡す
```

ReservationsDAO で「その人が今日どこに行く予定か」を調べ、座席を予約していれば seatId を使って SeatsDAO から座席の詳細（拠点・フロア・エリア・座席番号）を取得する、という**2段階の問い合わせ**になっています。JSP側（nameSearch.jsp ・ departmentSearch.jsp）では、受け取った seat が null でなければ seat.getBaseName() などを表示するだけです。

ここを直すとき: 検索結果に表示する項目をさらに増やしたい場合は、同じ要領で「Logicで必要なデータをDAOから集めて row に詰める」→「JSPで row.get(...) して表示する」という形を真似すれば追加できます。

5-3b. 拠点ごとのフロアマップ画像を切り替える仕組み（OFFICE_IMAGES）

座席利用状況・代理予約・マスタ更新の各画面では、「拠点（オフィス）」を選ぶとそのオフィスのフロアマップ画像が表示されます。この対応関係は dao/SeatsDAO.java の OFFICE_IMAGES という Map にまとめられています。

```
static final Map<String, String> OFFICE_IMAGES = new LinkedHashMap<>();

static {
    OFFICE_IMAGES.put("三田オフィス", "office_mita.png");
    OFFICE_IMAGES.put("芝浦オフィス", "office_shibaura.png");
}

public String getOfficeImage(String baseName) {
    return OFFICE_IMAGES.get(baseName);
}
```

JSP側では、選ばれている拠点名で `getOfficeImage(selBase)` を呼び、戻ってきたファイル名を `` に埋め込むだけです。

```
<%
    String officeImage = (selBase != null && !selBase.isEmpty())
        ? new SeatsDAO().getOfficeImage(selBase) : null;
%>

<% if (officeImage != null) { %>
    のフロアマップ">
<% } %>
```

新しいオフィスを増やす手順（`SeatsDAO.java` の冒頭コメントにも同じ内容を記載しています）：

1. **SEATS**（座席データ）に新オフィス分の座席を追加する（マスタ更新画面の「②座席を追加する」から「+ 新しいオフィスを追加する」を選んでも可）
2. フロアマップ画像（PNGなど）を用意し、`webapp/images/` と Eclipse側 `WebContent/images/` の両方に置く
3. `OFFICE_IMAGES` に "オフィス名" → "ファイル名" の対応を1行追加する

逆に「オフィスを無くしたい（表示しないようにしたい）」ときは、**SEATS** から該当オフィス分の座席データを削除し、`OFFICE_IMAGES` からも対応行を削除します（実際に行ったのは「田町オフィス」を削除したケースです）。

ここを直すとき：オフィスを追加・削除したいときはこのセクションの手順どおりに `SeatsDAO.java` を編集します。画像を変えたいだけなら、同じファイル名で画像を上書きするだけでOK（コードの変更は不要）。

5-3c. 「拠点 → フロア → エリア」を順番に絞り込む仕組み（カスケード選択）

座席利用状況画面・代理予約画面では、「①拠点を選ぶ → ②その拠点にあるフロアだけが選べるようになる → ③そのフロアにあるエリアだけが選べるようになる」という、前の選択に応じて次の選択肢が変わる**カスケード選択**を採用しています。「すべての拠点」のような曖昧な状態を作らず、常に「○○オフィスの○○エリア」のように場所が一意に決まるようにするためです。

仕組みは難しくありません。Logic側（`ControlServlet.showSeatStatus` など）で、選ばれている値に応じて段階的にリストを用意するだけです。

```
boolean baseChosen = filterBase != null && !filterBase.isEmpty();
boolean floorChosen = baseChosen && filterFloor != null && !filterFloor.isEmpty();
boolean areaChosen = floorChosen && filterArea != null && !filterArea.isEmpty();

if (baseChosen) req.setAttribute("floors", seatsDAO.getFloors(filterBase));
if (floorChosen) req.setAttribute("areas", seatsDAO.getAreas(filterBase, filterFloor));

List<Seat> filteredSeats = areaChosen
    ? seatsDAO.filterSeats(filterBase, filterFloor, filterArea)
    : new ArrayList<>();
```

「拠点が選ばれていなければフロアの選択肢は用意しない」「フロアが選ばれていなければエリアの選択肢は用意しない」という順番がポイントです。JSP側は `<select onchange="this.form.submit()">` でプルダウンを変更するたびにフォームを自動送信し、`floors / areas` が `null` の間は `<select disabled>` にして選べないようにしています。

```
<select name="filterFloor" onchange="this.form.submit()" <%= (floors == null) ? "disabled" : "" %>>
    ...
</select>
```

ここを直すとき: 同じ「親を選ぶまで子は選べない」絞り込みを別の画面でも作りたいときは、Logic側で「選ばれているか」のbooleanを段階的に作り、それに応じてリストをセットする → JSP側は `disabled` とプルダウンの自動送信、という形を真似します。

5-3d. 管理者ログイン中だけ画面の色を変える仕組み（CSS変数の上書き）

管理者でログインしている間は、ヘッダーやボタンの基調色がティール系からインディゴ系に変わります。`webapp/css/style.css` の `:root` で色は **CSS変数**として定義されており（`--teal` `--teal-light` `--teal-dark`）、ボタンやヘッダーなどあらゆる場所がこの変数を参照しています。

```
:root {
    --teal: #00897B;
    --teal-light: #4DB6AC;
    --teal-dark: #00695C;
}

.header { background: var(--teal); }
.btn-primary { background: var(--teal); color: #fff; }
```

管理者用ヘッダー（`headerAdmin.jspf`）と一般用ヘッダー（`header.jspf`）の先頭で、ログイン中のユーザーが管理者なら、この変数を上書きする `<style>` を出力しています。

```
<% if (Boolean.TRUE.equals(session.getAttribute("isAdmin"))) { %>
<style>
    :root { --teal:#3949AB; --teal-light:#7986CB; --teal-dark:#283593; }
</style>
<% } %>
```

CSSの `:root` はどこに `<style>` を書いても文書全体に効くため、ヘッダーの中に書くだけで**ページ全体の色が一括で変わります**。個別の `.btn` や `.header` を1つずつ書き換える必要がないのが、CSS変数を使う一番のメリットです。

ここを直すとき: 配色そのものを変えたいときはこの `<style>` 内の3つの色コードを変更するだけ。「管理者だけでなく特定の部署だけ色を変えたい」など条件を変えたい場合は、`if` の条件式を書き換えます。

5-4. メニュー画面の集計表示（同部署の出勤人数・2週間分の予約状況）

メニュー画面の表示は `control/ControlServlet.java` の `showMenu` メソッドが担当しています（このアプリでは `MenuLogic` という専用クラスを作らず、コントローラの中で直接処理しています）。

```
// 同じ部署の本日の出勤人数・座席一覧
Account me = accDAO.findById(userId);
List<Account> deptMembers = accDAO.findByBId(me.getBId());

List<Map<String, Object>> deptSeatList = new ArrayList<>();
long officeCount = 0;
for (Account a : deptMembers) {
    List<Reservation> mRes = resDAO.findTodayByUserId(a.getUserId());
    if (mRes.isEmpty()) continue;
    Reservation r = mRes.get(0);
    if (!"出勤".equals(r.getGyosaki())) continue;
    officeCount++;

    Seat seat = r.getSeatId() > 0 ? seatDAO.findById(r.getSeatId()) : null;
    Map<String, Object> row = new HashMap<>();
    row.put("name", a.getName());
    row.put("seat", seat);
    deptSeatList.add(row);
}
req.setAttribute("deptOfficeCount", officeCount);
req.setAttribute("deptSeatList", deptSeatList);
```

「自分の部署のメンバー一覧を取得 → 一人ひとりの本日の予約を調べる → 出社 の人だけ人数を数えつつ、氏名と座席をリストに詰める」という流れです。人数だけでなく「誰が・どの座席にいるか」までその場で組み立てているのがポイントで、JSP側（`menu.jsp` / `adminMenu.jsp`）はこの `deptSeatList` を `for` で回して `row.get("name")` ・ `row.get("seat")` を表示するだけになっています。

2週間分の予約状況（管理者メニューのみ表示）も同じ考え方で、`LocalDate.now().plusDays(i)` で14日分の日付を作りながら、1日ごとに `findByDate` で予約を取得し、件数の集計と「氏名＋座席」の明細リスト（`people`）の両方を組み立てています。

```
for (int i = 0; i < 14; i++) {
    String date = LocalDate.now().plusDays(i).toString();
    List<Reservation> dayRes = resDAO.findByDate(date);

    long officeNum = dayRes.stream().filter(r -> "出社".equals(r.getGyosaki())).count();
    long seatNum    = dayRes.stream().filter(r -> r.getSeatId() > 0).count();

    // 出社・座席が確定している人の「氏名＋座席」明細（折りたたみ表示用）
    List<Map<String, Object>> people = new ArrayList<>();
    for (Reservation r : dayRes) {
        if (!"出社".equals(r.getGyosaki()) || r.getSeatId() <= 0) continue;
        Account acc = accDAO.findById(r.getUserId());
        Seat seat = seatDAO.findById(r.getSeatId());
        people.add(...氏名と座席を詰めたMap...);
    }
    row.put("people", people);
    ...
}
```

`adminMenu.jsp` 側では、この `people` を `<details class="day-summary"><summary>...</summary><div class="day-detail">...</div></details>` という **HTML標準の折りたたみ要素** で包み、「閉じた状態は日付と件数だけのコンパクト表示」「クリックして開くと氏名・座席まで見える詳細表示」を、JavaScriptなしで実現しています（5-1のプルダウンメニューと同じ `<details>` / `<summary>` パターン）。

ここを直すとき：

- 件数の数え方や表示する範囲（2週間→1ヶ月など）を変えたい → `showMenu` メソッドの集計部分を編集
- 表示するレイアウトを変えたい → `menu.jsp`（一般） / `adminMenu.jsp`（管理者）の該当する `<div class="card">` を編集
- 一般ユーザーにも2週間分を見せたい → `if (isAdmin) { ... }` の条件を外し、`menu.jsp` 側にも同じ表示ブロックを追加する

6. よくある「ここを直したい」Q&A

Q. ボタンの表示文言を変えたい → 該当する `.jsp` ファイルを開いて、文字列を直接書き換えればOK。

Q. ログインできるユーザーを増やしたい → `dao/AccountsDAO.java` を開き、`ACCOUNTS` の中身 (`Arrays.asList( ... )` の引数) に `new Account("user006", "pass006", "氏名", 部署ID, 0)` のような行を1つ追加する (末尾の要素の後にカンマ `,` を付け忘れないこと)。 ※ `ACCOUNTS` は `Arrays.asList(...)` で作られた **サイズ変更不可のリスト** なので、`ACCOUNTS.add(...)` を呼び出すと `UnsupportedOperationException` で落ちます。必ず `Arrays.asList(...)` の引数リストの中身を直接編集してください。

Q. 「土日は予約できない」などのルールを追加したい → `model/ReserveLogic.java` の処理の中に、日付を判定する `if` 文を追加。

Q. 新しい画面を追加したい → 次の順番で作成します。

1. 必要なら `entity` にデータの入れ物を追加
2. `dao` にデータ取得用のメソッドを追加 (モックでOK)
3. `model` に処理クラス (`Logic` を実装) を作成
4. `control/ControlServlet.java` に `case` を追加
5. `webapp/WEB-INF/jsp` に画面 (JSP) を作成

関連ドキュメント:

- [02 DB接続ガイド.md](#) — モックデータを実際のMySQLに繋ぎ変える方法
- [03 研修ハンズオン手順書.md](#) — 研修当日の手順まとめ